

Curso de Programación de bajo nivel en Rust

Proyecto

René Manzano

25 de junio de 2026

Introducción

En este documento se describen los dos proyectos prácticos que se deben realizar como parte del curso de Rust. Además, se incluye una guía para publicar el código fuente y los ejecutables en GitHub, de modo que el público pueda acceder y ejecutar las aplicaciones.

1. Proyecto 1: Modificación del evaluador de operaciones aritméticas

El primer proyecto consiste en tomar el código del evaluador de expresiones aritméticas que hemos trabajado en clase y modificar su implementación. Actualmente, dicho código utiliza el puntero inteligente `Box` para almacenar los nodos del árbol de sintaxis abstracta (AST). El objetivo es reemplazar `Box` por alguna otra alternativa de gestión de memoria, como:

- `Rc` (referencia contada) para permitir el uso compartido del nodo.
- `Arc` (referencia contada atómica) si se requiere concurrencia.
- `RefCell` combinado con `Rc` para mutabilidad interior.
- Incluso se podría rediseñar usando referencias con `lifetimes`, aunque es más complejo.

El estudiante debe justificar la elección de la nueva estrategia, explicar las ventajas y desventajas respecto a `Box`, y asegurarse de que el evaluador siga funcionando correctamente.

2. Proyecto 2: Aplicación relacionada con la ingeniería

El segundo proyecto consiste en desarrollar una aplicación que esté vinculada con el área de trabajo o carrera del profesor (en este caso, ingeniería). Se deben emplear los siguientes conceptos de Rust:

- Patrones y `match`
- Deestructuración de estructuras y enumeraciones

- Préstamo (*borrowing*) y referencias
- Enumeraciones (`enum`) y estructuras (`struct`)
- Funciones y módulos

2.1. Ideas de proyectos

Se proponen dos opciones; el estudiante puede elegir una o plantear otra similar, siempre que cumpla con los requisitos técnicos.

2.1.1. Idea 1: Calculadora científica

Desarrollar una calculadora que permita operaciones básicas (suma, resta, multiplicación, división) y funciones científicas como seno, coseno, exponencial, logaritmo, raíz cuadrada, etc. La interfaz puede ser por consola o interactiva. Se debe modelar la expresión mediante un `enum` que represente los distintos tipos de operaciones, y usar `match` para evaluar. Además, se deben manejar correctamente las referencias para evitar copias innecesarias.

2.1.2. Idea 2: Modelado de un ticket (del capítulo 3 de «100 ejercicios de Rust»)

Se trata de implementar un sistema de tickets para un servicio de soporte o ventas. Cada ticket tiene un identificador, descripción, prioridad, estado (abierto, en progreso, cerrado) y posiblemente un usuario asignado. Se pueden definir `structs` para el ticket y `enums` para el estado y la prioridad. El programa debe permitir crear, modificar y listar tickets, aplicando patrones de coincidencia para manejar las diferentes acciones. Este ejercicio está inspirado en el capítulo 3 del libro «100 ejercicios de Rust».

Ambas opciones son adecuadas para ingenieros, ya que implican el manejo de datos estructurados, lógica de control y gestión de recursos, habilidades esenciales en el desarrollo de software de ingeniería.

2.2. Requisitos técnicos comunes

Independientemente de la opción elegida, el código deberá:

- Estar organizado en módulos (por ejemplo, `operaciones`, `modelos`, `main`).
- Utilizar `enums` para representar variantes de datos.
- Usar `match` y destructuración para extraer valores.
- Emplear referencias (`&`) para pasar datos sin moverlos.
- Incluir funciones que devuelvan `Result` para manejar errores.
- Estar documentado con comentarios (uso de `///` para documentación).

3. Guía para publicar en GitHub

A continuación se detallan los pasos para subir el código y publicar un ejecutable para que cualquier usuario pueda correr la aplicación.

3.1. Crear cuenta y repositorio

1. Si no se tiene, crear una cuenta en GitHub.
2. Iniciar sesión y hacer clic en el botón «New repository».
3. Asignar un nombre (por ejemplo, `rust-calculadora` o `rust-tickets`).
4. Elegir si será público o privado (para este curso se recomienda público).
5. No inicializar con README, .gitignore o licencia (se pueden añadir después).
6. Hacer clic en «Create repository».

3.2. Subir el código desde la computadora

Suponiendo que el proyecto ya está desarrollado localmente:

1. Abrir una terminal en la carpeta del proyecto.
2. Inicializar un repositorio Git local:

```
git init
```

3. Agregar todos los archivos:

```
git add .
```

4. Hacer el primer commit:

```
git commit -m "Primera versión del proyecto"
```

5. Conectar con el repositorio remoto (sustituir `USUARIO` y `REPO`):

```
git remote add origin https://github.com/USUARIO/REPO.git
```

6. Subir los cambios a la rama principal:

```
git branch -M main  
git push -u origin main
```

3.3. Publicar el ejecutable con GitHub Releases

Para que el público pueda descargar y ejecutar la aplicación sin necesidad de compilar, se recomienda usar la funcionalidad de *Releases* de GitHub:

1. Compilar el proyecto en modo «release»:

```
cargo build --release
```

2. Localizar el ejecutable generado en `target/release/` (el nombre dependerá del proyecto).
3. En la página del repositorio en GitHub, ir a la pestaña «Releases».
4. Hacer clic en «Draft a new release».
5. Crear una etiqueta (tag) para la versión (ej. `v1.0.0`) y un título descriptivo.
6. Adjuntar el archivo ejecutable (y si es necesario, otros archivos como `.dll` o `.so` en Windows/Linux, aunque Rust compila estáticamente, puede ser suficiente con el binario).
7. Publicar la release.
8. Los usuarios podrán descargar el ejecutable desde la sección de «Assets» de la release.

Es recomendable incluir un archivo `README.md` con instrucciones de uso y los requisitos del sistema.

Entrega

Lo ideal es entregar:

- El enlace al repositorio de GitHub (debe ser público).
- Un breve informe en PDF que describa las decisiones de diseño, las dificultades encontradas y las soluciones implementadas.
- El ejecutable publicado en la release será considerado para la evaluación de funcionalidad.

Como mencioné ayer, también pueden compartirme el código dentro de la carpeta source por correo. Pueden hacer un zip de todo el proyecto y mandarlo.

Muchas gracias.